

SIMULAÇÃO E CONSTRUÇÃO DE UMA UNIDADE LÓGICA E ARITMÉTICA (ULA) EM PYTHON

Rayanne da Silva Lima de França

Curso de Análise e Desenvolvimento de Sistemas (ADS)

UNINASSAU – Paulista Centro

RESUMO

Este trabalho apresenta o desenvolvimento e a simulação de uma Unidade Lógica e Aritmética (ULA) de 4 bits utilizando a linguagem de programação Python. O sistema foi projetado para demonstrar de forma prática as operações fundamentais que ocorrem no núcleo de uma Unidade Central de Processamento (CPU). Através de uma estrutura clara, o software é capaz de executar operações aritméticas básicas, como soma e subtração, e operações lógicas, incluindo AND, OR e também o XOR, além do Seletor de operações (Opcode) juntamente com o Mux (multiplexador) um tipo de caixa calculadora ou chave seletora para os cálculos das operações. Os resultados obtidos nos testes confirmaram a precisão e a estabilidade do modelo, validando assim o resultado esperado para cada operação e servindo como uma ferramenta didática para o estudo de Arquitetura de Computadores.

1 INTRODUÇÃO

A arquitetura de computadores compreende o estudo da estrutura, organização e interações entre os componentes de hardware e software que constituem um sistema computacional moderno. Como elemento central e indispensável dessa estrutura, encontra-se a Unidade Central de Processamento (CPU), a qual depende diretamente de subcircuitos para realizar o processamento de dados. A Unidade Lógica e Aritmética (ULA) é responsável por processar e executar todas as instruções matemáticas e lógicas previstas pelos programas, transformando comandos complexos de alto nível em sinais básicos de bits.

Compreender o funcionamento prático e estrutural de uma ULA é um passo fundamental para entender o domínio completo do fluxo de dados e o controle em sistemas digitais microprocessados. Diante disso, este artigo documenta detalhadamente o desenvolvimento de um simulador funcional de ULA em Python. Esta modelagem permite visualizar a integração e o chaveamento interno de dados que ocorrem em nível de hardware por meio de uma abordagem puramente lógica em software.

2 METODOLOGIA

O simulador da ULA foi totalmente codificado na linguagem Python, representando de maneira lógica e sequencial o comportamento intrínseco de circuitos eletrônicos combinacionais.

Ou seja, o sistema funciona perfeitamente a partir do recebimento de duas entradas de dados principais (A e B) e de um código de operação específico (opcode), que determina com precisão qual circuito interno ou sub-rotina será ativado através de uma estrutura de controle condicional.

Para a simulação lógica e aritmética, foram distribuídos e especificados os seguintes componentes essenciais dentro da aplicação:

- **Operações Aritméticas:** Implementação das funções de soma (+) e subtração (-), simulando de maneira clara os caminhos de dados binários percorridos internamente no processador.
- **Operações Lógicas:** Execução clássica das principais portas lógicas utilizando os operadores nativos do Python, contemplando o operador "E" para o AND lógico (que retorna 1 apenas quando ambos os bits são 1), o OR lógico (que retorna 1 se qualquer bit for 1) e a porta XOR para detecção de alternância bit a bit.
- **Seletor e Multiplexador (Mux):** A presença crucial do Seletor de operações (Opcode) operando juntamente com o Mux (multiplexador), atuando como um tipo de caixa calculadora ou chave seletora estrutural para isolar e direcionar os resultados das operações.
- **Sinalizadores (Flags de status):** Elementos integrados no encerramento de cada bloco operacional para sinalizar estados específicos obtidos nas saídas, como carry, zero ou overflow.

3 RESULTADOS

A fase de validação do simulador foi realizada por meio de testes práticos exaustivos, introduzindo sistematicamente diferentes valores de entrada e comandos operacionais no terminal. Nas rotinas aritméticas de soma e subtração, todas as saídas geradas pelo software corresponderam exatamente aos cálculos matemáticos esperados pela álgebra tradicional e aritmética binária de 4 bits.

Da mesma forma, as rotinas associadas às operações lógicas de AND e OR processaram corretamente as strings binárias dos números fornecidos, comprovando de forma empírica que a estrutura condicional desenvolvida em Python reflete com clareza o comportamento físico real dos circuitos lógicos de uma CPU. O multiplexador chaveou com sucesso os canais de dados sem gerar falhas de tipagem ou concorrência operacional de memória.

4 CONCLUSÃO

A construção do simulador cumpriu com todos os objetivos educacionais e práticos propostos inicialmente, demonstrando que a ULA atua perfeitamente como algo de extrema importância no meio computacional e estrutural da CPU. A flexibilidade obtida no controle do

software permitiu a mudança dos dados, o que, por exemplo, tornou possível visualizar de forma clara e linear o fluxo de dados completo que ocorre desde os comandos emitidos pelo Sistema Operacional até a entrega final do resultado processado à memória física.

O projeto cumpriu seu papel prático e consolidou de forma experimental exemplos consolidados de conceitos complexos de arquitetura de computadores, abrindo portas para que os desenvolvedores expandam as aplicações diretas desse conhecimento conceitual no design e na modelagem de novos hardwares virtuais e microcontroladores dedicados.

5 REFERÊNCIAS BIBLIOGRÁFICAS

SILVA, Rodrigo M. da; ADAMCZEWSKI, Tiago A. **ULA de 2 entradas de 4 bits e 4 operações**. 2010. Relatório Acadêmico (Disciplina de Projeto de Circuitos Integrados Digitais) – Universidade Federal do Paraná, Curitiba, 2010.